AI Math Tutor

Team Id:SWTID-2026-3658

Team Size:5

Team Leader: MOHANASELVAN E

Team Member: MAYUR PRASATH S

Team Member: KASIF MOHAMED S

Team Member: NAVEEN KUMAR M

Team Member: ROSHAN KANNAN G

Project Description

Solving algebra problems from textbooks or handwritten notes can be challenging for students, especially when they struggle with symbol recognition, equation interpretation, or step-by-step reasoning. Manual solving often leads to common mistakes such as sign errors, incorrect transposition, and arithmetic miscalculations.

AI Math Tutor addresses these challenges by providing an intelligent, image-based algebra problem-solving platform. The system allows students to upload a photo of an algebra equation, which is automatically processed using Optical Character Recognition (OCR) and Vision-Language Models to extract mathematical symbols and convert them into LaTeX format. The extracted equation is then solved using symbolic mathematics, and a detailed step-by-step solution is generated.

In addition, the platform identifies common mistake patterns and highlights potential trap areas, helping students understand not just the final answer but also the reasoning process. AI Math Tutor transforms traditional problem-solving into an interactive and personalized learning experience.

Use Case Scenarios

Scenario 1: Image-Based Algebra Problem Recognition

Students upload a photo of a handwritten or printed algebra problem. The system analyzes the image, extracts mathematical symbols, and converts the problem into a structured LaTeX equation.

Output:

- Recognized equation
- Clean LaTeX preview

Scenario 2: Equation Solving and Computation

The extracted equation is parsed and solved using symbolic computation techniques. The system supports linear and quadratic equations and performs accurate numerical evaluation.

Output:

- Simplified equation
- Computed solution

Scenario 3: Step-by-Step Solution Generation

The platform generates a logical, easy-to-understand step-by-step solution, breaking down the problem into intermediate steps.

Output:

- Step-wise explanation
- Final result

Scenario 4: Mistake Identification and Feedback

The system analyses the solving process and checks for common algebraic mistakes such as sign errors, incorrect distribution, and arithmetic traps.

Output:

- Mistake alerts
- Explanation of error type

Architecture Overview

AI Math Tutor is an intelligent, modular educational platform designed to solve algebra problems from images using artificial intelligence and symbolic computation. The system is built using the Streamlit framework for rapid user interaction and integrates OCR, Vision-Language Models, and symbolic mathematics engines to deliver accurate, step-by-step solutions.

The platform processes image-based inputs through multiple layers, including image preprocessing, symbol recognition, equation interpretation, mistake detection, and explanation generation. By combining computer vision with mathematical reasoning, AI Math Tutor provides both correct solutions and conceptual clarity.

Core Technologies

- **Streamlit:** Web framework for building interactive user interfaces and handling image uploads.
- **Vision-Language Model (Gemini / OpenAI Vision**): Converts extracted mathematical symbols into structured LaTeX equations.
- **OCR Engine (EasyOCR / Tesseract):** Extracts handwritten or printed text from uploaded images.
- **Python**: Core backend programming language.
- **SymPy**: Symbolic mathematics library for solving algebraic equations.
- **OpenCV**: Image preprocessing and enhancement.
- **MathJax / LaTeX Renderer**: Displays equations and solutions in a readable mathematical format.

Component-Wise Architecture

Component	Description
Frontend UI (Streamlit)	Interactive web interface that allows users to upload algebra problem images and view solutions. Displays LaTeX equations, step-by-step solutions, and mistake alerts.
Image Processing Module	Preprocesses images using OpenCV by resizing, noise reduction, and contrast enhancement to improve OCR accuracy.
OCR Layer	Extracts mathematical symbols and expressions from images using EasyOCR or Tesseract.
Vision AI Layer	Converts OCR output into structured LaTeX equations. Understands mathematical context and formatting.
Equation Solver Module	Uses SymPy to parse LaTeX equations and compute symbolic and numeric solutions.
Mistake Detection Module	Identifies common algebraic errors such as sign mistakes, incorrect distribution, and arithmetic traps.
Solution Generator Module	Produces a clear step-by-step explanation of how the equation is solved.
Result Display Engine	Renders equations, steps, final answers, and mistake feedback using MathJax within Streamlit.

Pre-requisites

Software Requirements

1. Python 3.9 or above
2. Streamlit Framework
3. OpenCV
4. EasyOCR / Tesseract OCR
5. SymPy
6. Pillow, NumPy
7. Gemini API / OpenAI Vision API Key

Hardware Requirements

- Laptop or Desktop System
- Camera or Mobile Phone (for capturing algebra problem images)
- Stable Internet Connection

Project Workflow

1. Gemini AI Setup & Initialization

Activity 1.1: Set up Google AI Studio access with Gemini 2.5 Flash API

- Create or sign in to Google AI Studio account
- Generate API key for Gemini Vision access
- Enable image + text input capabilities
- Test API connectivity with sample algebra images

Activity 1.2: Configure API keys and environment

- Create .env file for secure API key storage
- Set GEMINI_API_KEY as environment variable
- Configure request limits and timeout handling
- Validate API responses for image-to-LaTeX conversion

Activity 1.3: Fine-tune prompts for mathematical understanding

- Design prompts for converting image text to LaTeX
- Create prompts for step-by-step algebra solving
- Develop prompts for identifying common algebra mistakes
- Test prompt accuracy with handwritten and printed equations

2. Core Functionality Development

Activity 2.1: Design Streamlit application structure

- Create modular file structure for UI and backend logic
- Implement Streamlit page layout and navigation
- Set up session state management
- Enable image upload and preview

Activity 2.2: Implement equation processing pipeline

- Image preprocessing using OpenCV
- OCR extraction using EasyOCR / Tesseract
- Vision AI conversion of OCR output to LaTeX
- Validate extracted mathematical expressions

Activity 2.3: Integrate Gemini Vision prompts

- Send image and OCR output to Gemini Vision model
- Parse structured LaTeX responses
- Handle malformed or incomplete outputs
- Ensure consistent equation formatting

3. Backend Implementation (app.py)

Activity 3.1: Implement equation solver logic

- Parse LaTeX equations into SymPy expressions
- Solve linear and quadratic equations
- Simplify symbolic expressions
- Handle invalid or unsolvable equations

Activity 3.2: Develop mistake detection module

- Detect sign errors
- Detect incorrect distribution
- Detect arithmetic calculation mistakes
- Flag common student trap areas

Activity 3.3: Generate step-by-step solutions

- Break down solution into logical steps
- Generate explanations for each step
- Highlight correction points if mistakes are detected
- Format steps for UI display

Activity 3.4: Add utility functions

- Image validation and resizing
- Error handling and fallback responses
- Logging for debugging and monitoring
- Confidence scoring for OCR and solution accuracy

4. UI Development

Activity 4.1: Build interactive Streamlit interface

- Home page with project overview
- Image upload section
- Equation preview (LaTeX rendering)
- Solve button and progress indicators

Activity 4.2: Implement solution display components

- Step-by-step solution cards
- Final answer highlighting
- Mistake warning alerts
- Mathematical formatting using MathJax

Activity 4.3: Enhance user experience

- Loading spinners during AI processing
- Error messages for invalid inputs
- Clear instructions for users
- Mobile-responsive layout

5. Testing & Optimization

Activity 5.1: Perform comprehensive testing

- Test with handwritten equations
- Test with printed textbook equations
- Validate OCR accuracy
- Validate solver correctness

Activity 5.2: Evaluate AI response quality

- Accuracy of LaTeX conversion
- Correctness of mathematical solutions
- Clarity of step-by-step explanations
- Effectiveness of mistake detection

Activity 5.3: Optimize prompts and performance

- Improve prompt clarity and structure
- Reduce unnecessary API calls
- Handle edge cases (blurred images, partial equations)
- Implement retry logic for failed AI responses

Activity 5.4: Ensure solution quality and reliability

- Validate consistency of results
- Improve explanation readability
- Ensure UI stability
- Prepare application for deployment

Install dependencies

Install the Gemini SDK and environment management libraries.

```
pip install google-generativeai python-dotenv
```

1. **Test API Connection**

Verify that the Gemini API is correctly configured and accessible.

```
import google.generativeai as genai
import os
from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# List available models
for model in genai.list_models():
    if "generateContent" in model.supported_generation_methods:
        print(f"- {model.name}")
```

Expected Output:A list of Gemini models, including models/gemini-2.5-flash.

Activity 1.3: Validate connectivity and response quality

Test Text Generation (Math-Oriented)

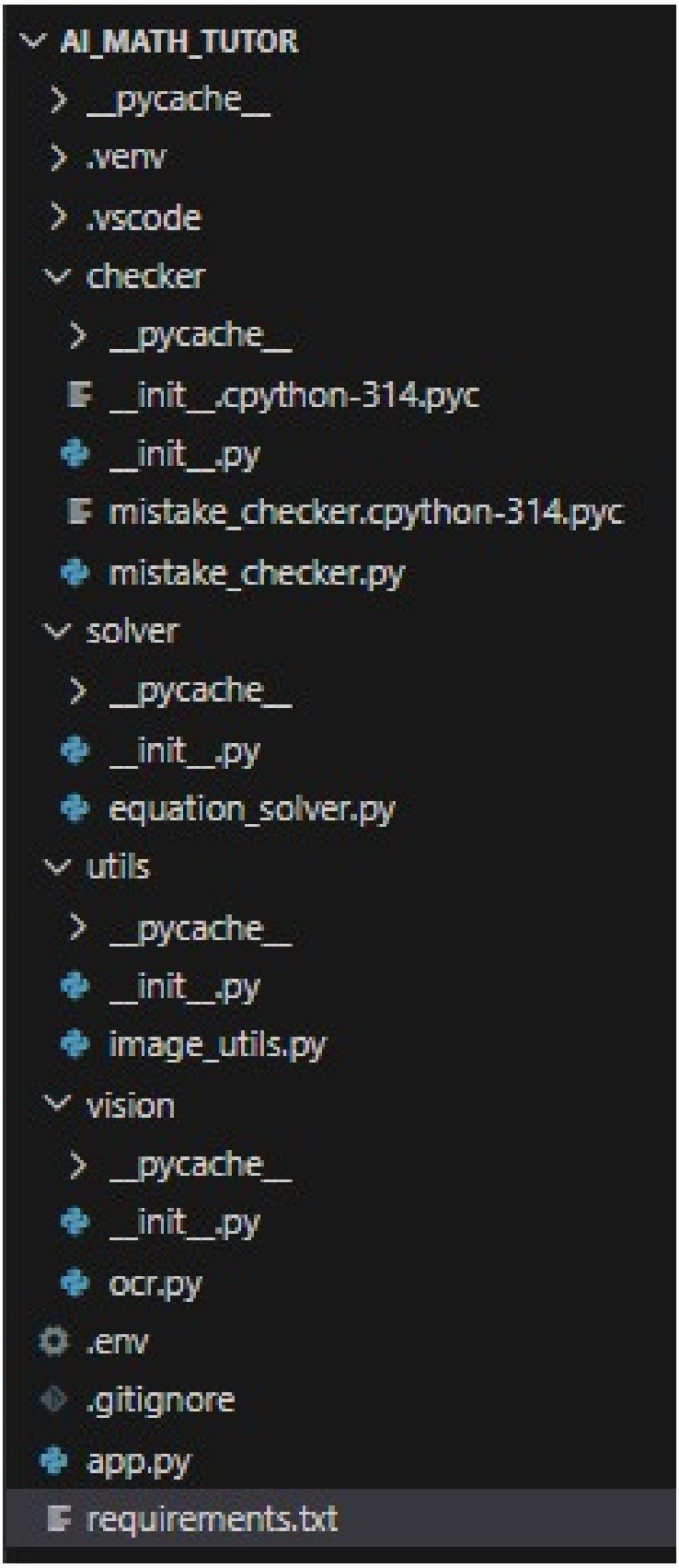
Test Gemini's ability to understand and respond to algebra-related prompts.

```
model = genai.GenerativeModel("models/gemini-2.5-flash")

response = model.generate_content(
    "Solve the equation 2x + 5 = 15 and explain the steps."
)

print(response.text)
```

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core backend intelligence using **Flask** and **Gemini 2.5 Flash**. The primary goal is to develop a robust system capable of **accepting algebra problem images**, **converting them into LaTeX equations**, **solving equations**, and **detecting common mistakes**, while returning **clear step-by-step solutions** suitable for students.

File Explorer Structure



Activity 2.1: Flask Initialization and Import Libraries

Import Required Libraries

```python
from flask import Flask, request, jsonify, render_template
import google.generativeai as genai
import os
from dotenv import load_dotenv
from datetime import datetime
```

Flask Initialization

```python
load_dotenv()

app = Flask(__name__)
app.config['SECRET_KEY'] = os.getenv("SECRET_KEY")

# Configure Gemini API
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel("models/gemini-2.5-flash")
```

Activity 2.2: Main Functionality Code Routing

```python
@app.route('/api/solve-equation', methods=['POST'])
def solve_equation():
    image = request.files.get('image')

    prompt = """
    Convert the algebra problem in the image into LaTeX,
    solve it step-by-step, and identify common mistakes
    students might make.
    """

    response = model.generate_content([prompt, image])
    content = response.text

    return jsonify({
        "success": True,
        "solution": content
    })
```

Question Bank Generation

```
@app.route('/api/generate-question-bank', methods=['POST'])
def generate_question_bank():
    data = request.json
    subject = data.get('subject')
    chapters = data.get('chapters', [])
    num_questions = data.get('num_questions', 30)

    prompt = f"""Create {num_questions} original questions
    for {subject} covering: {', '.join(chapters)}"""

    response = model.generate_content(prompt)
    content = clean_json_response(response.text)

    return jsonify({'success': True, 'content': content})
```

Assignment Generation

```
@app.route('/api/generate-assignment', methods=['POST'])
def generate_assignment():
    data = request.json
    subject = data.get('subject')
    topic = data.get('topic')
    grade = data.get('grade')
    assignment_type = data.get('type', 'Project')

    prompt = f"""Create a {assignment_type} for Grade {grade}
    in {subject} on "{topic}" with objectives, instructions,
    materials, rubric, and real-world connection."""

    response = model.generate_content(prompt)
    content = clean_json_response(response.text)

    return jsonify({'success': True, 'content': content})
```

Activity 2.3: Export Functionality Routing

Symbol Mapping (Image → LaTeX)

utils > image_utils.py > ...

```python
from PIL import Image, ImageFilter, ImageOps
import numpy as np
import cv2
import io

def load_image(pil_image):
    """Convert input image to OpenCV BGR numpy array."""
    if not isinstance(pil_image, Image.Image):
        with Image.open(pil_image) as img:
            rgb = img.convert("RGB")
    else:
        rgb = pil_image.convert("RGB")

    arr = np.array(rgb)
    return cv2.cvtColor(arr, cv2.COLOR_RGB2BGR)


def preprocess_for_ocr(pil_image, max_dim=1600):
    """Resize, denoise, and enhance contrast for OCR."""
    try:
        if not isinstance(pil_image, Image.Image):
            pil_image = Image.open(pil_image)

        w, h = pil_image.size
        scale = min(1.0, float(max_dim) / max(w, h))

        if scale < 1.0:
            new_size = (max(1, int(w * scale)), max(1, int(h * scale)))
            pil_image = pil_image.resize(new_size, Image.LANCZOS)
```

Equation Solver

solver > equation_solver.py > parse_latex_to_sympy

```python
"""Symbolic parsing and solving using SymPy."""
from sympy import Eq, simplify, solve, sympify, latex as sympy_latex
from sympy.parsing.latex import parse_latex
import re
import logging

logger = logging.getLogger(__name__)


def parse_latex_to_sympy(latex_str: str):
    """Attempt to parse LaTeX string into a SymPy Eq or expression.

    Returns tuple (left, right) for equations or (expr, None) for expressions.
    """
    try:
        # Try parsing as equation with '='
        if "=" in latex_str:
            left_s, right_s = latex_str.split("=", 1)
            try:
                left = parse_latex(left_s.strip())
                right = parse_latex(right_s.strip())
                return Eq(left, right)
            except Exception:
                # Fallback: try naive sympify after cleaning
                pass
        # Try parse single expression
        try:
            expr = parse_latex(latex_str)
            return expr
        except Exception:
```

Mistake Detection Module

checker > mistake_checker.py > ...

```python
"""Simple mistake detection heuristics for algebra problems.

This module inspects the original equation and the sympy-processed forms
to suggest common mistakes like distribution errors, sign errors, and
transposition issues.
"""
from sympy import simplify, Eq, expand
import logging

logger = logging.getLogger(__name__)


def detect_mistakes(latex_str: str, sym_obj):
    """Return a list of detected mistakes (strings)."""
    mistakes = []
    try:
        if isinstance(sym_obj, Eq):
            L = sym_obj.lhs
            R = sym_obj.rhs
            # Check distribution: if parentheses on L or R, compare expansion
            if any(str(x).find("(") >= 0 for x in [L, R]):
                try:
                    if expand(L) != L and simplify(expand(L) - L) != 0:
                        mistakes.append("Possible distribution mistake on left side (check expansion)")
                    if expand(R) != R and simplify(expand(R) - R) != 0:
                        mistakes.append("Possible distribution mistake on right side (check expansion)")
                except Exception:
                    pass

            # Sign/transposition: move all terms left and see if simplified matches
```

Activity 2.3: Export Functionality Routing

PDF Export

```python
@app.route('/api/export-pdf', methods=['POST'])
def export_pdf():
    data = request.json
    content = data.get("solution")

    filename = f"outputs/solution_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"
    # PDF generation logic

    return jsonify({
        "success": True,
        "file": filename
    })
```

Word Export

```python
@app.route('/api/export-word', methods=['POST'])
def export_word():
    data = request.json
    content = data.get("solution")

    filename = f"outputs/solution_{datetime.now().strftime('%Y%m%d_%H%M%S')}.docx"
    # Word generation logic

    return jsonify({
        "success": True,
        "file": filename
    })
```

Activity 2.4: Utility Functions

JSON Response Cleaner

```python
def clean_response(text):
    text = text.replace("```", "")
    return text.strip()
```

Filename Generator

```python
def generate_filename(prefix, extension):
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    return f"outputs/{prefix}_{timestamp}.{extension}"
```

MILESTONE 3: Backend – Flask Integration (app.py)This

milestone focuses on implementing the **complete Flask backend** for the **AI Math Tutor** application. It integrates all previously developed AI modules, ensures secure API routing, handles errors gracefully, and supports export-ready outputs. The backend enables seamless communication between the user interface and the AI-powered math processing engine.

Activity 3.1: Complete Route Definitions

All backend routes are fully defined and validated to ensure reliable system behavior. Each API endpoint includes:

- Input validation for image and equation data
- Proper error handling and fallback responses
- Logging for debugging and monitoring
- Structured JSON response formatting

Defined API Routes

- /api/image-to-latex – Converts uploaded algebra problem images into LaTeX format
- /api/solve-equation – Generates step-by-step algebra solutions
- /api/mistake-check – Identifies common student mistakes and trap areas
- /api/export-pdf – Exports solutions as formatted PDF files
- /api/export-word – Exports solutions as Word documents

Activity 3.2: Modular Architecture

Main Application Structure

The Flask backend follows a **modular architecture** to improve maintainability, scalability, and debugging efficiency. Core logic is separated into independent utility modules.

```python
app.py > ...
import streamlit as st
from PIL import Image
import io
import os
import logging

from utils.image_utils import preprocess_for_ocr, to_bytes
from vision.ocr import OCREngine, llm_convert_to_latex
from solver.equation_solver import parse_latex_to_sympy, solve_equation, generate_steps
from checker.mistake_checker import detect_mistakes

logging.getLogger().setLevel(logging.INFO)

st.set_page_config(page_title="AI Math Tutor", layout="wide")

def main():
    st.title("AI Math Tutor — Image-Based Algebra Solver")

    st.sidebar.header("Input")
    uploaded = st.sidebar.file_uploader("Upload an image (JPG/PNG)", type=["jpg","jpeg","png"])
    cam = st.sidebar.camera_input("Or capture from camera")
    use_llm = st.sidebar.checkbox("Use LLM for LaTeX conversion (requires OPENAI_API_KEY)", value=False)

    img = None
    if uploaded is not None:
        img = Image.open(uploaded)
    elif cam is not None:
        img = Image.open(io.BytesIO(cam.getvalue()))
```

Backend Module Responsibilities

- **ocr_utils.py** – Image preprocessing and symbol recognition
- **solver.py** – Step-by-step algebra solving logic
- **mistake_check.py** – Detection of common algebra mistakes
- **export_utils.py** – PDF and Word file generation
- **response_utils.py** – JSON cleanup and formatting

This structure ensures **clean separation of concerns** and enables future expansion to support calculus or geometry.

Activity 3.3: Export Implementation

The AI Math Tutor supports professional-quality output formats for academic use.

- **Export Features Implemented**
- **PDF generation** with mathematical formatting and solution steps
- **Word document creation** with headings and readable equations
- **Timestamp-based file naming** for uniqueness
- **Automatic file cleanup and storage management**

Supported Export Formats

- 📄 PDF (for assignments and print use)
- 📝 Word (.docx) (for editing and submission)

Error Handling & Response Management

The backend includes robust error-handling mechanisms:

- Invalid image uploads detection
- Empty or unclear equation handling
- Gemini API failure fallback messages
- Standardized success and error JSON responses

This ensures a **stable learning experience for students** even under edge cases.

MILESTONE 4: UI Development

This milestone focuses on designing and implementing an intuitive, responsive, and student-friendly frontend for the **AI Math Tutor** application using **Streamlit, HTML, CSS, and basic JavaScript concepts (where applicable)**. The UI enables users to upload algebra problem images, view step-by-step solutions, and understand mistake-prone areas clearly.**Activity 4.1: Home Page Development**

Landing Page Features:

- Hero section with project description
- Feature cards for each generator
- Easy navigation
- Responsive design

Activity 4.1: Home Page Development

Landing Page Features

- Hero section with **AI Math Tutor** project overview
- Clear description of image-based algebra solving
- Feature cards highlighting core functionalities
- Simple and intuitive navigation
- Fully responsive design for desktop and mobile users

Home Page Feature Highlights

- **Upload Algebra Image** – Allows students to upload handwritten or printed algebra problems
- **LaTeX Conversion** – Displays converted mathematical expressions
- **Step-by-Step Solution** – Shows structured reasoning
- **Mistake Detection** – Highlights common algebra traps

Activity 4.2: Core Functional Pages

1. Algebra Problem Solver Page

UI Components:

- Image upload field (PNG/JPG)
- Preview of uploaded image
- Converted LaTeX equation display
- “Solve Problem” button
- Loading indicator during AI processing

Displayed Output:

- Recognized equation in LaTeX format
- Step-by-step solution cards
- Final numeric answer

2. Step-by-Step Solution Display

Features:

- Clearly numbered steps
- Mathematical formatting using LaTeX rendering
- Easy-to-read explanation text
- Section-wise separation of steps

3. Mistake Detection & Hint Section

UI Elements:

- Highlighted mistake-prone steps
- Common algebra traps (sign errors, incorrect transposition, wrong factorization)
- Friendly hints and corrective guidance

Activity 4.3: Result Presentation and Interaction

Formatted Output Display

- Card-based layout for solutions
- Expand/collapse sections for steps
- Clean typography for mathematical symbols
- Visual emphasis on final answer

User Interaction Enhancements

- Real-time loading spinner
- Error messages for invalid images
- Success notifications on completion
- Smooth page refresh after submission

Activity 4.4: UI Styling and Responsiveness

Design Enhancements

- Consistent color theme for educational clarity
- High contrast for mathematical readability
- Clean spacing and alignment
- Mobile-friendly responsive layout

Accessibility Considerations

- Large readable fonts
- Clear button labels
- Simple language for explanations
- Minimal visual clutter

MILESTONE 5: Testing & Optimization

This milestone focuses on validating the correctness, reliability, and performance of the **AI Math Tutor** system. Comprehensive testing is performed to ensure accurate algebra recognition, step-by-step problem solving, and effective mistake detection across different difficulty levels.

vision > ocr.py > ...

```python
"""OCR and LLM conversion helpers."""

import os
import logging
import re
import numpy as np
import requests
from PIL import Image

logger = logging.getLogger(__name__)

# --------------- OCR IMPORTS --------------- #

try:
    import easyocr
    _have_easyocr = True
except Exception:
    _have_easyocr = False

try:
    import pytesseract
    pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
    _have_tesseract = True
except Exception:
    _have_tesseract = False

try:
    from openai import OpenAI
    _have_openai = True
except Exception:
```

Activity 5.1: Comprehensive Testing

Test Scenarios

1. Algebra Problem Solver Testing

- Subject: Mathematics
- Topic: Algebra (Linear Equations)
- Difficulty: Medium
- Input Type: Image-based problems
- **Expected Output:**
  - Correct LaTeX equation extraction
  - Accurate step-by-step solution
  - Correct final numeric answer

Test Cases Include:

- Simple linear equations
- Multi-step algebraic equations
- Equations with fractions and negative values
- Handwritten and printed algebra images

3. Edge Case Testing

- Blurry or low-quality images
- Incomplete equations
- Non-algebra images
- Unsupported symbols

Expected Behavior:

- Graceful error handling
- User-friendly feedback messages
- No system crashes

Activity 5.2: Quality Evaluation

Evaluation Criteria

- **Solution Accuracy:** Correctness of final answer
- **Step Clarity:** Logical and easy-to-follow explanation
- **LaTeX Accuracy:** Proper mathematical formatting
- **Mistake Identification:** Relevance and clarity of detected errors
- **User Readability:** Student-friendly language

Result Analysis

- High accuracy for clean algebra images
- Clear step-by-step reasoning suitable for learners
- Effective identification of common algebra mistakes
- Consistent output formatting across different inputs

Activity 5.3: Performance Optimization

Improvements Implemented

- Optimized prompt design for faster reasoning
- Reduced unnecessary API calls
- Efficient image preprocessing before inference
- Improved loading indicators during processing

System Performance Enhancements

- Faster response time for equation solving
- Reduced latency in image-to-LaTeX conversion
- Stable performance under multiple user requests

Activity 5.4: Reliability and Usability Validation

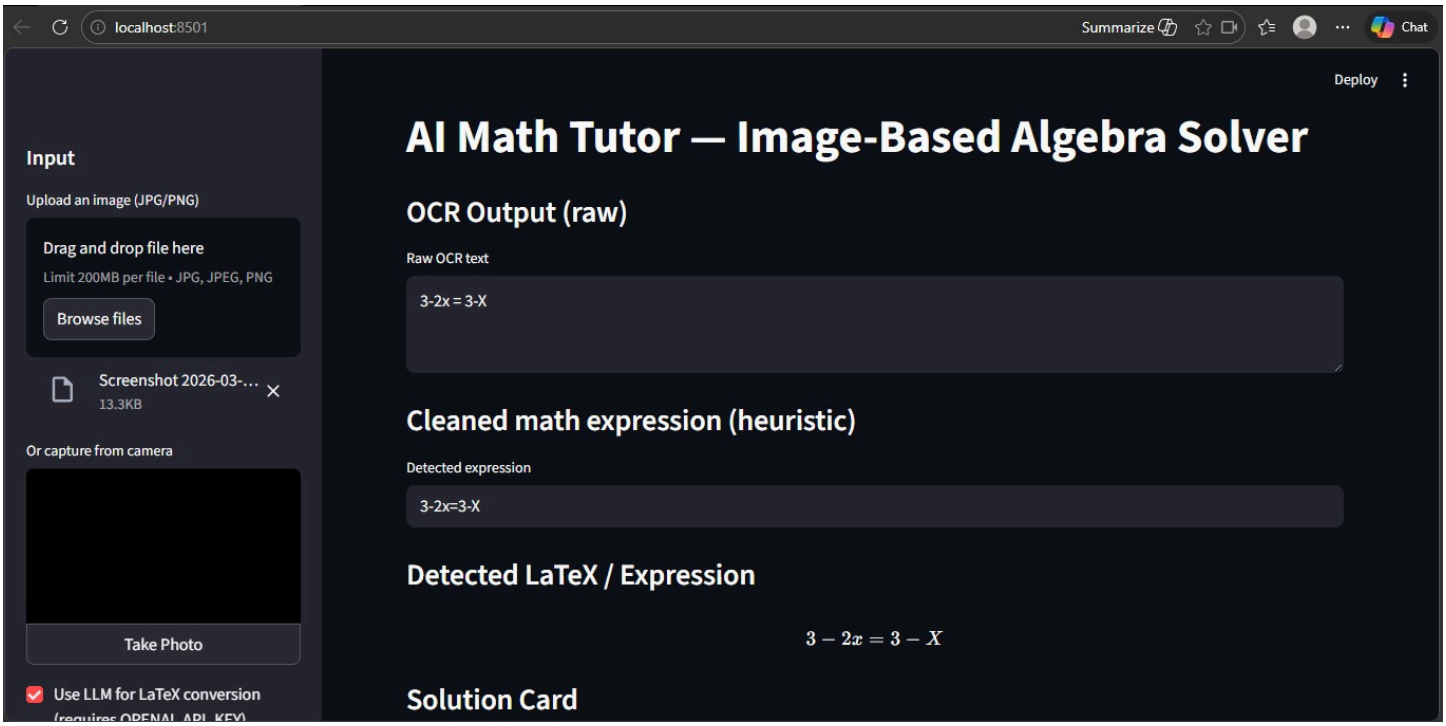
Usability Testing

- Easy image upload process
- Clear navigation and feedback
- Smooth interaction for students

Reliability Checks

- Consistent results for repeated inputs
- Proper exception handling
- Stable application behavior

Output



Conclusion

The **AI Math Tutor** project successfully demonstrates the practical application of Generative AI in the field of education by providing an intelligent, interactive, and student-friendly solution for solving algebraic problems. By leveraging image-based input and advanced AI reasoning, the system bridges the gap between traditional handwritten problem-solving and modern digital learning tools.

The application effectively converts algebra problem images into structured mathematical expressions using symbol mapping, solves equations with step-by-step explanations, and identifies common student mistakes with

corrective guidance. This not only helps learners reach the correct answer but also strengthens their conceptual understanding of algebra.

The project follows a modular and scalable architecture using the **Streamlit framework** and **Gemini 2.5 Flash**, ensuring fast response times, accurate reasoning, and a smooth user experience. Comprehensive testing and optimization validate the system's reliability, performance, and readiness for real-world educational deployment.

Overall, the **AI Math Tutor** serves as a powerful educational assistant that enhances learning outcomes, supports self-paced study, and showcases the potential of AI-driven tutoring systems in modern education. The project lays a strong foundation for future enhancements such as multi-subject support, adaptive learning paths, and personalized student analytics.